HTTP Requests and Responses

Learning Objective: Explain GET and POST methods and distinguish between 2xx, 4xx, and 5xx response status categories, with a focus on API debugging practices.

Target Level: Beginner	**Difficulty:** Intermediate	**Est. Duration:** 45 mins (Max: 45m)	**Status:** IN_REVIEW

1. Introduction (3 mins)

HTTP is the language of the web. Every time you load a webpage, submit a form, or use a mobile application, your device acts as a client sending an HTTP request to a server, which replies with an HTTP response. In this lesson, you will learn how the two most common request methods—GET and POST—work, how to interpret HTTP status code categories (2xx, 4xx, and 5xx), and how to apply these concepts to debug real-world APIs. By the end, you will be able to read a request/response exchange, identify what went wrong, and decide on the next debugging step. This lesson is grounded in [RFC 9110: HTTP Semantics] and [MDN Web Docs].

[Applied QC Fix - Factual]:
Replace with: '403 Forbidden (server understands the request but refuses to authorize it; authentication may or may not have occurred)'

2. Prerequisites (3 mins)

Before starting, you should understand:\n- The basic client-server model: a client (browser or app) requests resources from a server.\n- What a URL is and how to navigate to one in a web browser.\n- That data sent over the web is often formatted as JSON or plain text.\nNo prior programming experience is required, though familiarity with a terminal or browser Developer Tools will help during the hands-on activity.

3. Key Concepts (6 mins)

Core Terminology\n- **HTTP Request**: A message from client to server containing a method (GET, POST), a target URL, headers, and optionally a body.\n- **HTTP Response**: A message from server to client containing a status code, headers, and optionally a body.\n- **GET**: Requests a representation of a resource. It is *safe* (read-only) and *idempotent* (repeating it has the same effect). Parameters are typically sent in the query string.\n- **POST**: Submits data to be processed by the server (e.g., creating a resource). It is not idempotent and typically carries a request body.\n- **Status Code Classes**:\n - **2xx Success**: The request was received, understood, and accepted.\n - **4xx Client Error**: The request contains bad syntax or cannot be fulfilled due to a client issue.\n - **5xx Server Error**: The server failed to fulfill a valid request.\n- **API Debugging**: The practice of inspecting requests and responses to isolate whether a problem lies in the client request, the network, or the server.

4. Detailed Explanations (10 mins)

GET Method Deep Dive\nGET is used to retrieve data. According to [RFC 9110: HTTP Semantics], GET is defined as safe and idempotent, meaning it should not change server state and repeating it yields the same result. Query parameters appear after `?` in the URL (e.g., `GET /users?id=42`). Responses are often cacheable.\n\n## POST Method Deep Dive\nPOST is used to submit data to a resource, commonly to create a new record. Unlike GET, POST is not idempotent: sending the same POST twice may create two resources. The request body carries the payload, and the `Content-Type` header tells the server how to parse it (e.g., `application/json`).\n\n## Status Code Categories\n- **2xx Success**: `200 OK` (request succeeded), `201 Created` (resource created), `204 No Content` (success, no body).\n- **4xx Client Error**: `400 Bad Request` (malformed request), `401 Unauthorized` (authentication required), `403 Forbidden` (authenticated but not allowed), `404 Not Found` (resource missing), `422 Unprocessable Entity` (semantic errors), `429 Too Many Requests` (rate limited).\n- **5xx Server Error**: `500 Internal Server Error` (unexpected server condition), `502 Bad Gateway` (upstream error), `503 Service Unavailable` (server overloaded or down).\n\n## API Debugging Practices\n1. **Check the status code first**: 2xx means proceed; 4xx means fix the client request; 5xx means investigate the server.\n2. **Read the response body**: APIs often include human-readable error messages in JSON.\n3. **Inspect request headers**: Verify `Authorization`, `Content-Type`, and `Accept`.\n4. **Use browser DevTools**: Open the Network tab, reproduce the issue, and examine the request/response headers and payload.\n5. **Use curl with verbose mode**: `curl -v` shows the full handshake, headers, and status line.\n6. **Isolate variables**: Change one part of the request at a time (URL, method, headers, body) to identify the cause.\n\nSources: [RFC 9110: HTTP Semantics], [MDN Web Docs], [HTTP: The Definitive Guide].

5. Practical Examples (7 mins)

Example 1: Successful GET\nRequest:\n```\nGET /api/users/1 HTTP/1.1\nHost: api.example.com\nAccept: application/json\n```\nResponse:\n```\nHTTP/1.1 200 OK\nContent-Type: application/json\n\n{"id":1,"name":"Alice"}\n```\nDebugging takeaway: 200 OK means the client request was valid and the server returned the resource.\n\n### Example 2: Successful POST\nRequest:\n```\nPOST /api/users HTTP/1.1\nHost: api.example.com\nContent-Type: application/json\n\n{"name":"Bob","email":"bob@example.com"}\n```\nResponse:\n```\nHTTP/1.1 201 Created\nContent-Type: application/json\nLocation: /api/users/2\n\n{"id":2,"name":"Bob"}\n```\nDebugging takeaway: 201 Created confirms the resource was successfully created.\n\n### Example 3: Client Error (404)\nRequest:\n```\nGET /api/user/1 HTTP/1.1\n```\nResponse:\n```\nHTTP/1.1 404 Not Found\n\n{"error":"User not found"}\n```\nDebugging takeaway: 4xx indicates a client-side issue—check the URL spelling and resource ID.\n\n### Example 4: Server Error (500)\nResponse:\n```\nHTTP/1.1 500 Internal Server Error\n\n{"error":"Something went wrong"}\n```\nDebugging takeaway: 5xx indicates a server-side issue; check server logs or contact the API provider.

6. Real-World Applications (5 mins)

Modern web and mobile applications rely on RESTful APIs that use GET and POST extensively. For example:\n- **Social media feeds** use GET to retrieve posts.\n- **Signup forms** use POST to create user accounts.\n- **Payment gateways** (e.g., Stripe) return 2xx for successful charges, 4xx for invalid card details, and 5xx for processor outages.\n\nIn production debugging, engineers monitor status code distributions. A spike in 5xx errors triggers an incident; a spike in 4xx errors suggests a client-side bug or API contract change. Status codes also guide retry logic: retry on 5xx or 429 (with backoff), but do not blindly retry 4xx errors like 400 or 404.

7. Hands-On Activity (6 mins)

Activity: Inspect a Live API with curl and DevTools\nWe will use the free public test API at JSONPlaceholder.\n\n**Step 1: GET request in the browser**\n1. Open https://jsonplaceholder.typicode.com/posts/1 in your browser.\n2. Right-click → Inspect → Network tab.\n3. Refresh the page. Click the request to view headers and response.\n4. Note the method (GET), status code (200 OK), and response body.\n\n**Step 2: GET request with curl**\nRun in your terminal:\n```bash\ncurl -i https://jsonplaceholder.typicode.com/posts/1\n```\nObserve the `HTTP/1.1 200 OK` line and JSON body.\n\n**Step 3: POST request with curl**\nRun:\n```bash\ncurl -X POST -H "Content-Type: application/json" -d '{"title":"Hello","body":"World","userId":1}' https://jsonplaceholder.typicode.com/posts\n```\nObserve the `201 Created` response.\n\n**Step 4: Trigger a client error**\nRun:\n```bash\ncurl -i https://jsonplaceholder.typicode.com/posts/99999\n```\nObserve the `404 Not Found` response.\n\n**Step 5: Reflect**\nFor each response, identify the status class (2xx, 4xx, or 5xx) and write down one debugging action you would take next.

8. Knowledge Check (3 mins)

Question 1\nWhich HTTP method is typically used to retrieve a resource without modifying it?\n**Answer:** GET. It is safe and idempotent, designed only for reading data.\n\n### Question 2\nA `404 Not Found` response belongs to which status code category?\n**Answer:** 4xx Client Error. The client requested a resource that the server cannot locate.\n\n### Question 3\nYou call an API and receive a `500 Internal Server Error`. What is the most appropriate first debugging action?\n**Answer:** Treat it as a server-side issue. Check server logs or contact the API provider; do not assume the problem is in your request.\n\n### Question 4\nTrue or False: POST requests are idempotent.\n**Answer:** False. Repeating the same POST may create duplicate resources.\n\n### Question 5\nWhich status code indicates that a resource was successfully created?\n**Answer:** `201 Created`.

9. Summary (2 mins)

You now understand the HTTP request/response cycle. Use **GET** to retrieve data and **POST** to submit data. Interpret status codes by class: **2xx** means success, **4xx** means fix the client request, and **5xx** means investigate the server. Effective API debugging starts with the status code, continues with the response body and request headers, and uses tools like browser DevTools and `curl` to isolate problems.

10. Additional Resources

Recommended Resources\n- **YouTube**: [HTTP Crash Course & Status Codes Explained](https://www.youtube.com/watch?v=iYM2zFP3Zn0) by Traversy Media — visual walkthrough of HTTP methods and status codes.\n- **Book**: [HTTP: The Definitive Guide](https://books.google.com/books?id=f_a540C52z4C) by Gourley & Totty (O'Reilly) — comprehensive textbook reference on HTTP architecture, headers, and security.\n- **Documentation**: [MDN Web Docs: HTTP Methods](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods) and [RFC 9110: HTTP Semantics](https://www.rfc-editor.org/rfc/rfc9110.html).